Cluster Computing

Course Code: INT-315

Continuous Assessment – 2

Submitted in partial fulfillment of the requirements for the award of degree of

Bachelor of Technology in Computer Science Engineering,

Submitted to: Richa Nigam

LOVELY PROFESSIONAL UNIVERSITY, PHAGWARA, PUNJAB.

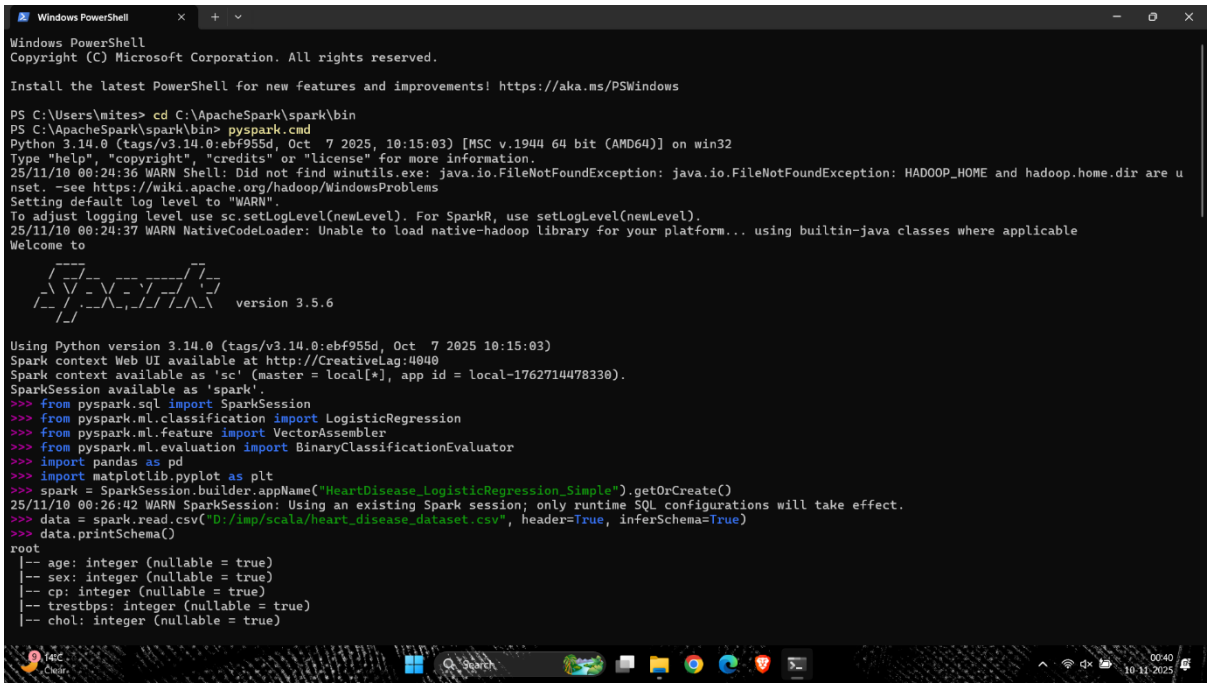
Submitted by: Mitesh Panda (12202392) (36)

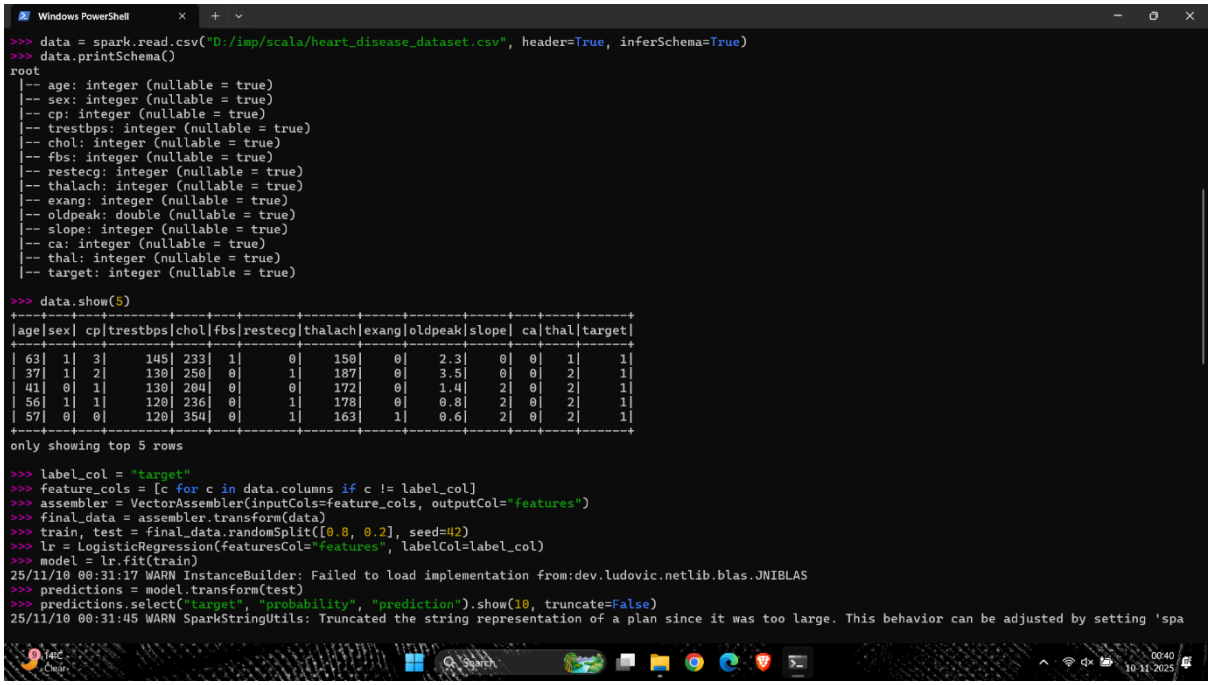
TABLE OF CONTENTS:

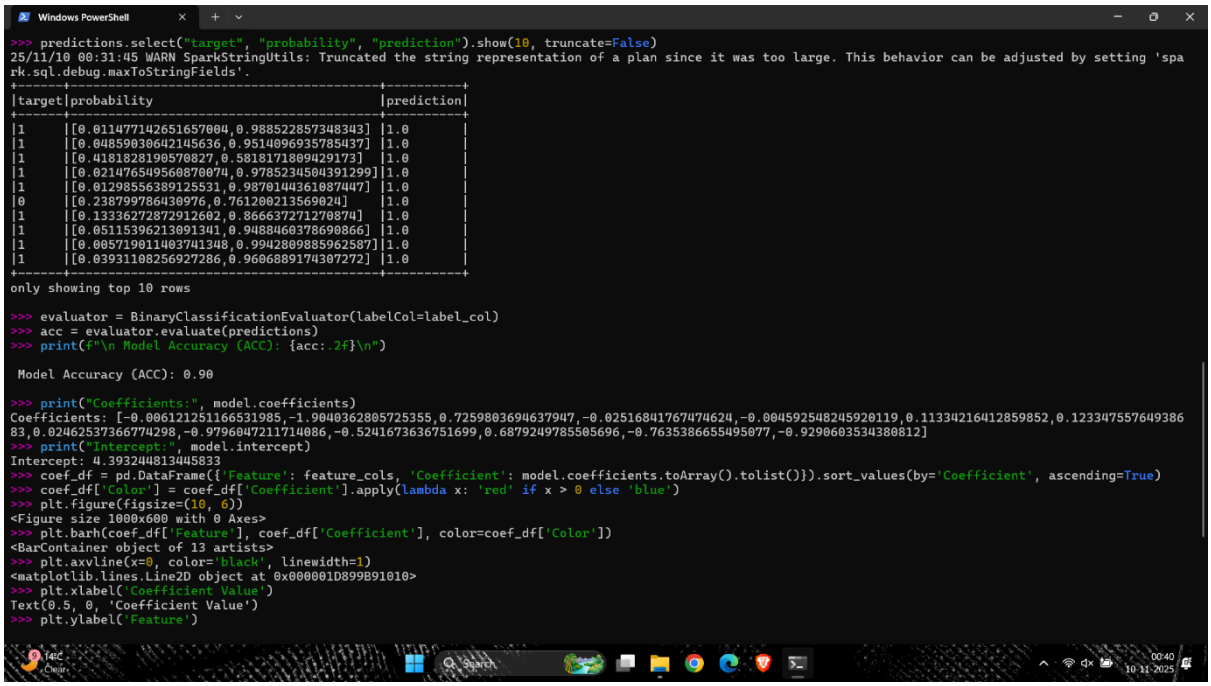
Introduction:

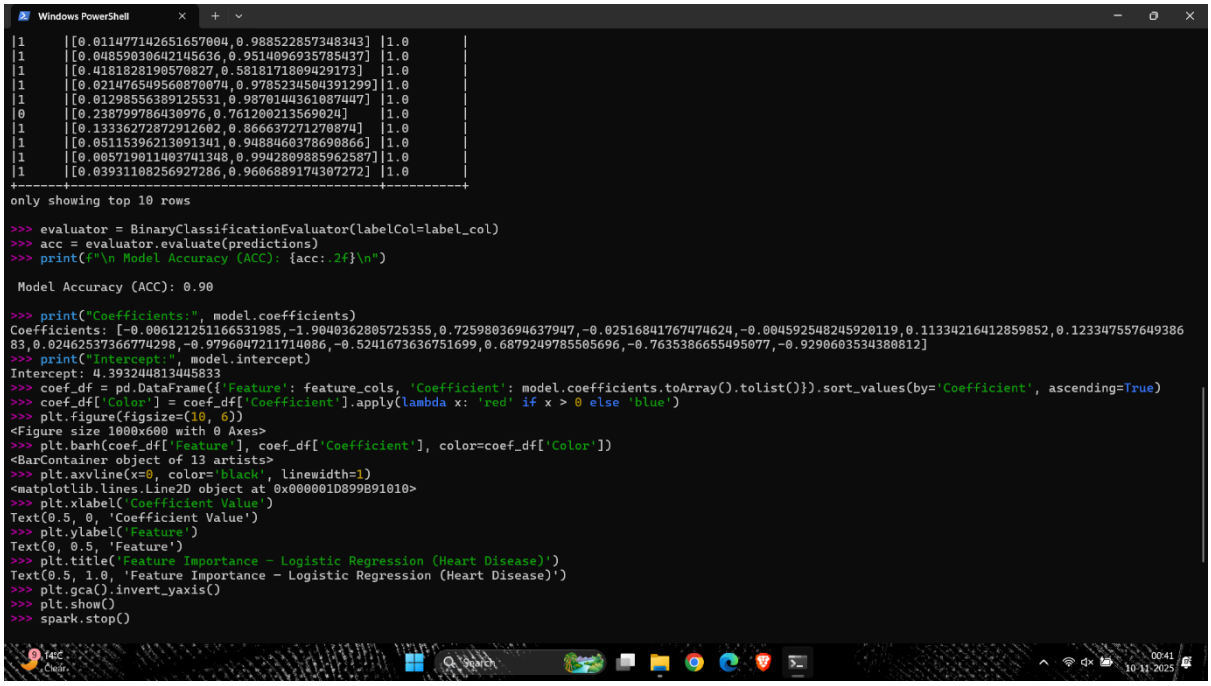
Logistic Regression is a widely used statistical and machine learning technique designed for predicting binary outcomes, where the dependent variable can take only two possible values such as “yes” or “no,” “1” or “0,” or “disease” and “no disease.” Unlike linear regression, which predicts continuous numerical values, logistic regression estimates the probability that a given input belongs to a particular class. It uses the logistic or sigmoid function to map predicted values into a range between 0 and 1, making it ideal for classification tasks. The model expresses the relationship between independent variables (such as age, cholesterol level, or blood pressure) and the dependent variable (such as presence of heart disease) through a linear combination of the input features, which is then passed through the sigmoid function to produce a probability. If the resulting probability is greater than or equal to a threshold (commonly 0.5), the output is classified as 1, otherwise as 0. Logistic regression is valued for its simplicity, efficiency, and interpretability, as each coefficient indicates how a specific feature influences the likelihood of the outcome. In this project, logistic regression was applied to predict the presence of heart disease based on various health indicators. The model not only provides accurate classification but also helps identify which factors most strongly contribute to the risk of heart disease, making it both a predictive and an explanatory tool in healthcare analytics.

Code Snippets:




Windows PowerShell
>>> data = spark.read.csv("D:/imp/scala/heart_disease_dataset.csv", header=True, inferSchema=True)
>>> data.printSchema()
root
 |-- age: integer (nullable = true)
 |-- sex: integer (nullable = true)
 |-- cp: integer (nullable = true)
 |-- trestbps: integer (nullable = true)
 |-- chol: integer (nullable = true)
 |-- fbs: integer (nullable = true)
 |-- restecg: integer (nullable = true)
 |-- thalach: integer (nullable = true)
 |-- exang: integer (nullable = true)
 |-- oldpeak: double (nullable = true)
 |-- slope: integer (nullable = true)
 |-- ca: integer (nullable = true)
 |-- thal: integer (nullable = true)
 |-- target: integer (nullable = true)
>>> data.show(5)
+---+---+---+--------+----+---+-------+-------+-----+-------+-----+---+----+------+
|age|sex| cp|trestbps|chol|fbs|restecg|thalach|exang|oldpeak|slope| ca|thal|target|
+---+---+---+--------+----+---+-------+-------+-----+-------+-----+---+----+------+
63	1	3	145	233	1	0	150	0	2.3	0	0	1	1
37	1	2	130	250	0	1	187	0	3.5	0	0	2	1
41	0	1	130	204	0	0	172	0	1.4	2	0	2	1
56	1	1	120	236	0	1	178	0	0.8	2	0	2	1
57	0	0	120	354	0	1	163	1	0.6	2	0	2	1
+---+---+---+--------+----+---+-------+-------+-----+-------+-----+---+----+------+
only showing top 5 rows
>>> label_col = "target"
>>> feature_cols = [c for c in data.columns if c != label_col]
>>> assembler = VectorAssembler(inputCols=feature_cols, outputCol="features")
>>> final_data = assembler.transform(data)
>>> train, test = final_data.randomSplit([0.8, 0.2], seed=42)
>>> lr = LogisticRegression(featuresCol="features", labelCol=label_col)
>>> model = lr.fit(train)
25/11/10 00:31:17 WARN InstanceBuilder: Failed to load implementation from:dev.ludovic.netlib.blas.JNIBLAS
>>> predictions = model.transform(test)
>>> predictions.select("target", "probability", "prediction").show(10, truncate=False)
25/11/10 00:31:45 WARN SparkStringUtils: Truncated the string representation of a plan since it was too large. This behavior can be adjusted by setting 'spa

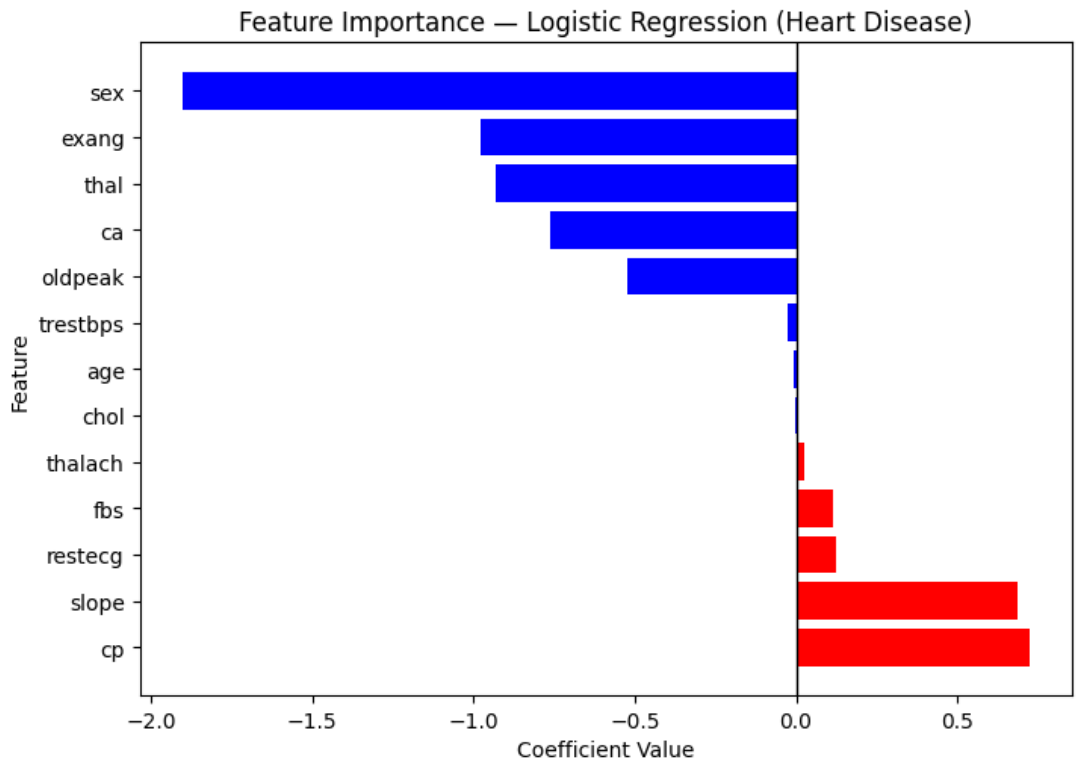

Windows PowerShell
>>> predictions.select("target", "probability", "prediction").show(10, truncate=False)
25/11/10 00:31:45 WARN SparkStringUtils: Truncated the string representation of a plan since it was too large. This behavior can be adjusted by setting 'spa
rk.sql.debug.maxToStringFields'.
+------+--+----------+
|target|probability |prediction|
+------+--+----------+
1	[0.011477142651657004,0.9885228573483343]	1.0
1	[0.04859030642145636,0.9514096935785437]	1.0
1	[0.4181828190570827,0.5818171809429173]	1.0
1	[0.021476549560870074,0.9785234504391299]	1.0
1	[0.012985563891255 31,0.9870144361087447]	1.0
0	[0.238799786430976,0.761200213569024]	1.0
1	[0.13336272872912602,0.8666372712708745]	1.0
1	[0.05115396213091341,0.9488460378690866]	1.0
1	[0.005719011403741348,0.9942809885962587]	1.0
1	[0.03931108256927286,0.9606889174307272]	1.0
+------+--+----------+
only showing top 10 rows
>>> evaluator = BinaryClassificationEvaluator(labelCol=label_col)
>>> acc = evaluator.evaluate(predictions)
>>> print(f"\n Model Accuracy (ACC): {acc:.2f}\n")
 Model Accuracy (ACC): 0.90
>>> print("Coefficients:", model.coefficients)
Coefficients: [-0.006121251166531985,-1.9040362805725355,0.7259803694637947,-0.025168417674746 24,-0.004592548245920119,0.11334216412859852,0.123347557649386
83,0.02462537366774298,-0.9796047211714086,-0.5241673636751699,0.6879249785505696,-0.7635386655495077,-0.9290603534380812]
>>> print("Intercept:", model.intercept)
Intercept: 4.393244813445833
>>> coef_df = pd.DataFrame({'Feature': feature_cols, 'Coefficient': model.coefficients.toArray().tolist()}).sort_values(by='Coefficient', ascending=True)
>>> coef_df['Color'] = coef_df['Coefficient'].apply(lambda x: 'red' if x > 0 else 'blue')
>>> plt.figure(figsize=(10, 6))
<Figure size 1000x600 with 0 Axes>
>>> plt.barh(coef_df['Feature'], coef_df['Coefficient'], color=coef_df['Color'])
<BarContainer object of 13 artists>
>>> plt.axvline(x=0, color='black', linewidth=1)
<matplotlib.lines.Line2D object at 0x000001D899B91010>
>>> plt.xlabel('Coefficient Value')
Text(0.5, 0, 'Coefficient Value')
>>> plt.ylabel('Feature')



Workflow:

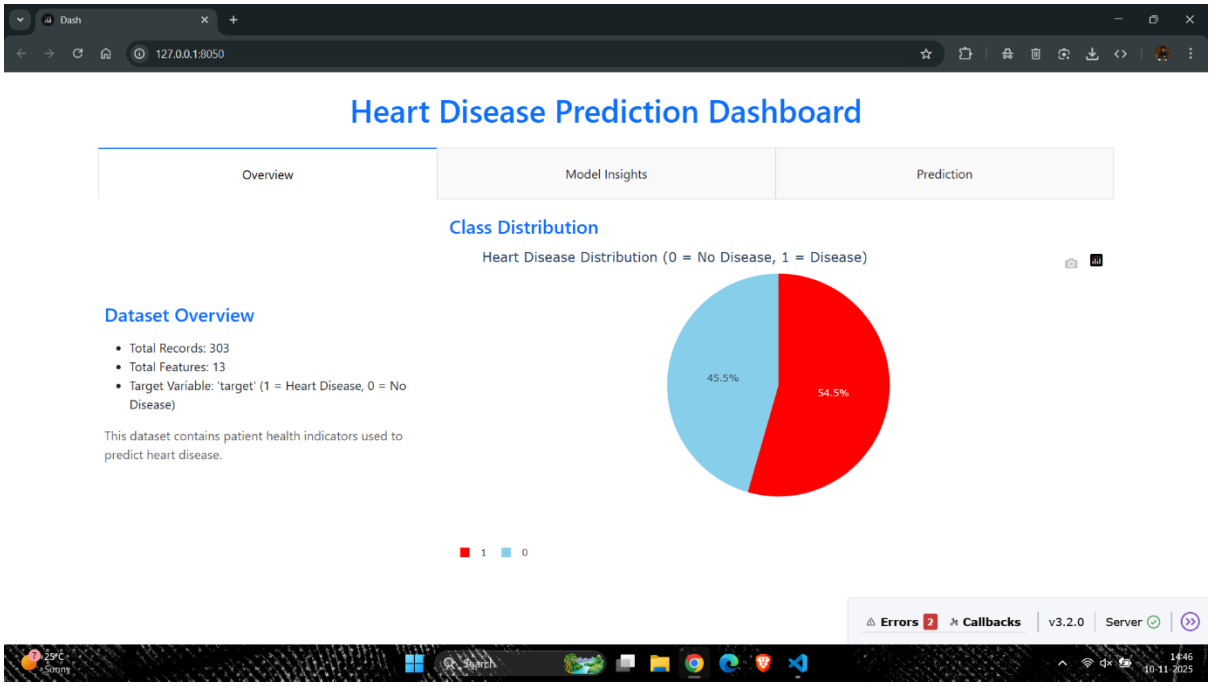
The workflow of this code demonstrates the implementation of Logistic Regression using PySpark to predict heart disease based on multiple health-related features. The process begins by creating a Spark session, which initializes the PySpark environment for distributed data processing. The heart disease dataset is then loaded using spark.read.csv() with headers and schema inference enabled. After verifying the data structure through printSchema() and show(), the feature columns (such as age, sex, cp, chol, etc.) are combined into a single feature vector using the VectorAssembler transformer, while the target column (target) is used as the label. The data is split into training and testing sets in an 80:20 ratio, and a logistic regression model is trained on the training data using the LogisticRegression() class. Predictions are generated on the test set, displaying the actual label, predicted probability, and final classification. The model's accuracy is evaluated using the BinaryClassificationEvaluator, which computes the Area Under Curve (AUC), showing a performance of approximately 0.90, indicating good predictive capability. The learned coefficients and intercept are then extracted to understand the weight and direction of influence of each feature on heart disease prediction. Finally, these coefficients are visualized in a color-coded bar chart using Matplotlib, where positive values (in red) indicate features that increase heart disease risk and negative values (in blue) indicate features that reduce it. This workflow effectively integrates PySpark's machine learning capabilities with Python's visualization tools to perform large-scale classification and interpret feature importance.

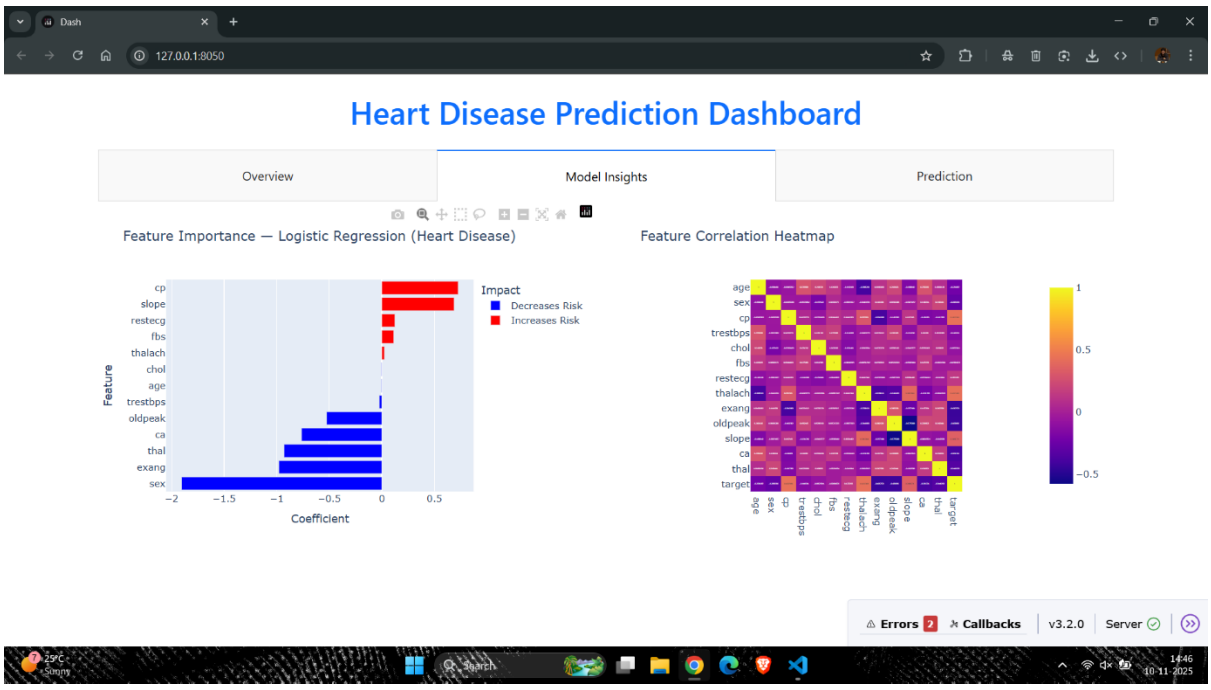
Bar Plot:

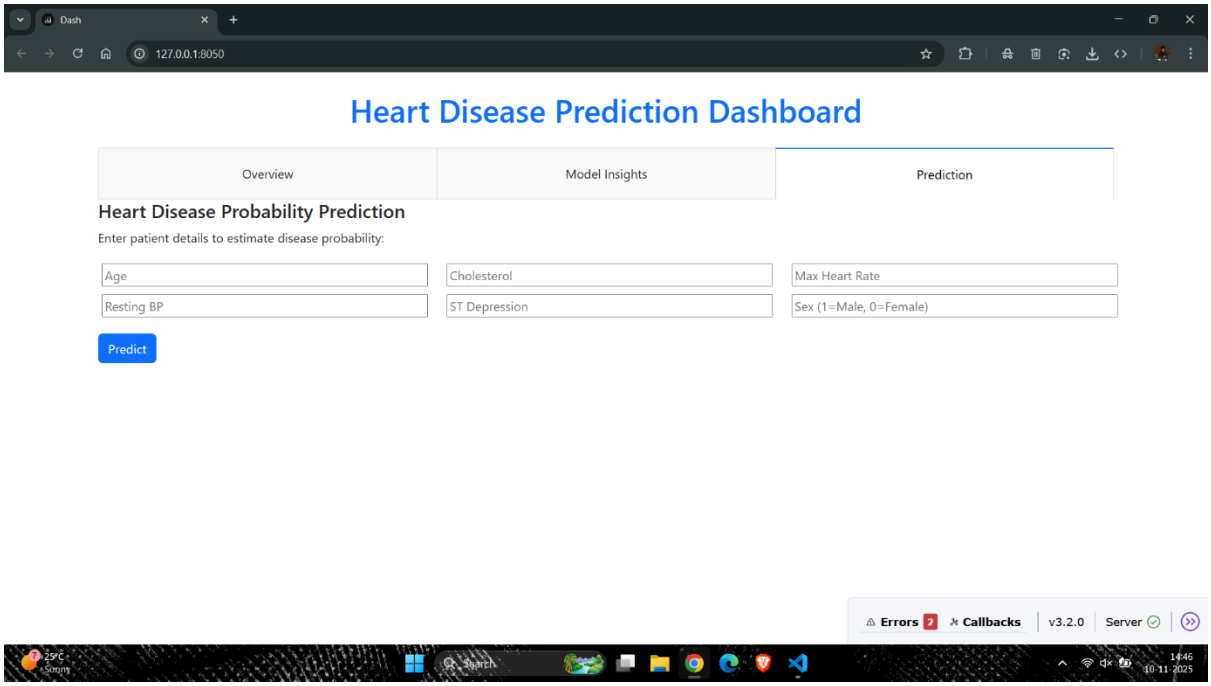


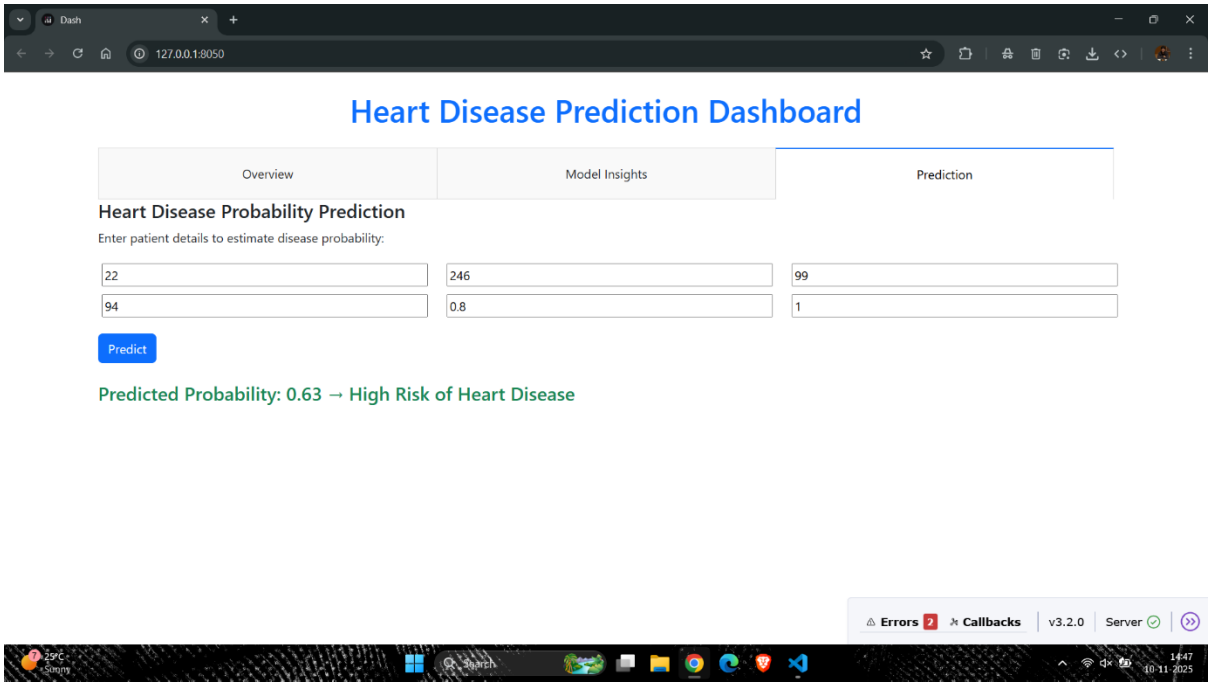
The feature importance plot illustrates how each variable influences the likelihood of heart disease as determined by the logistic regression model. The direction and magnitude of the coefficients indicate both the type and strength of association between each feature and the target outcome. Features with positive coefficients; shown in red, such as chest pain type (cp), slope of peak exercise ST segment (slope), and fasting blood sugar (fbs), have a direct relationship with the probability of heart disease meaning higher values for these features increase the likelihood of a positive diagnosis. Conversely, features with negative coefficients; shown in blue, such as sex, exercise-induced angina (exang), thalassemia (thal), number of major vessels (ca), and ST depression induced by exercise (oldpeak), exhibit an inverse relationship, suggesting that higher values for these factors decrease the likelihood of heart disease. The magnitude of each bar reflects the relative importance of the feature in the model, with chest pain type (cp) and sex emerging as the most influential predictors. Overall, the plot indicates that chest pain characteristics and ECG-related indicators play a significant role in determining heart disease risk, while demographic and exercise-related factors also contribute meaningfully to the prediction outcomes.

Dashboard:


Dash
127.0.0.1:8050
Heart Disease Prediction Dashboard
Overview
Model Insights
Prediction
Class Distribution
Heart Disease Distribution (0 = No Disease, 1 = Disease)
Dataset Overview
Total Records: 303
Total Features: 13
Target Variable: 'target' (1 = Heart Disease, 0 = No Disease)
This dataset contains patient health indicators used to predict heart disease.
45.5%
54.5%
1
0
Errors
Callbacks
v3.2.0
Server


Dash
127.0.0.1:8050
Heart Disease Prediction Dashboard
Overview
Model Insights
Prediction
Feature Importance — Logistic Regression (Heart Disease)
Feature Correlation Heatmap
Impact
Decreases Risk
Increases Risk
Feature
Coefficient
cp
slope
restecg
fbs
thalach
chol
age
trestbps
oldpeak
ca
thal
exang
sex
Errors
Callbacks
v3.2.0
Server


Dash
127.0.0.1:8050
Heart Disease Prediction Dashboard
Overview
Model Insights
Prediction
Heart Disease Probability Prediction
Enter patient details to estimate disease probability:
Age
Cholesterol
Max Heart Rate
Resting BP
ST Depression
Sex (1=Male, 0=Female)
Predict
Errors 2
Callbacks
v3.2.0
Server


Dash
127.0.0.1:8050
Heart Disease Prediction Dashboard
Overview
Model Insights
Prediction
Heart Disease Probability Prediction
Enter patient details to estimate disease probability:
22
246
99
94
0.8
1
Predict
Predicted Probability: 0.63 → High Risk of Heart Disease
Errors 2
Callbacks
v3.2.0
Server